

```

# -*- coding: utf-8 -*-
"""vfs_router_v5.3_gold_master.py"""

import hashlib
import os
import zlib
import struct
import time

# =====
# 1. DIRECTORY CONFIGURATION
# =====
try:
    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
except NameError:
    BASE_DIR = os.getcwd()

SOURCE_DIR = os.path.join(BASE_DIR, "raw_data")
VAULT_DIR = os.path.join(BASE_DIR, "SOVEREIGN_VFS_VAULT")

os.makedirs(SOURCE_DIR, exist_ok=True)
os.makedirs(VAULT_DIR, exist_ok=True)

# =====
# 2. CORE VFS ENGINE (V5.3 GOLD MASTER)
# =====
class VFSSovereignArchitect:
    def __init__(self):
        self.magic_sig = b"VFS!"
        self.sector_size = 4096

    def _get_slab_size(self, size):
        GB, TB, PB = 1024**3, 1024**4, 1024**5
        if size < GB: return 65536
        if size < TB: return 1048576
        if size < PB: return 67108864
        return 1073741824

    def trigger_save(self, path, ram_data, tier_limit, tier_name, storage_mode):
        orig_size = len(ram_data)
        if orig_size == 0:
            print("\n[!] ERROR: Input is 0 bytes. Aborting."); return

        start_time = time.perf_counter()

```

```

start_time = time.perf_counter()
cumulative_cpu_time = 0.0
slab_size = self._get_slab_size(orig_size)
level = 1 if storage_mode == "HOT" else 9
chunks = []
cumulative_entropy = 0
master_hasher = hashlib.sha256()

for i in range(0, orig_size, slab_size):
    chunk = ram_data[i : i + slab_size]
    master_hasher.update(chunk)
    c_hash = hashlib.sha256(chunk).digest()

    cpu_start = time.process_time()
    compressed_payload = zlib.compress(chunk, level=level)
    cpu_end = time.process_time()

    cumulative_cpu_time += (cpu_end - cpu_start)
    chunk_entropy_pct = (len(compressed_payload) / len(chunk)) * 100.0
    cumulative_entropy += chunk_entropy_pct

    logical_payload = compressed_payload
    logical_len = len(logical_payload)

    padding_needed = (self.sector_size - (logical_len % self.sector_size)) % self.sector_size
    physical_payload = logical_payload + (b'\x00' * padding_needed)

    chunks.append({
        "physical_payload": physical_payload,
        "logical_len": logical_len,
        "hash": c_hash,
        "tag": b'Z'
    })

avg_ent = cumulative_entropy / len(chunks)
map_tax = (len(chunks) * 45 * 2) + 64
logical_v_size = 4 + sum(x['logical_len'] for x in chunks) + map_tax
net_saving = orig_size - logical_v_size
is_efficient = (avg_ent <= tier_limit and net_saving > 0)
route = "VFS (Vault)" if is_efficient else "DIRECT (Raw)"

tmp_path = path + ".tmp"
try:
    with open(tmp_path, "wb") as f:
        if is_efficient:
            f.write(self.magic_sig)
            map_metadata_raw = b""

```

```

        for x in chunks:
            pos = f.tell()
            f.write(x['physical_payload'])
            map_metadata_raw += struct.pack(">QI32sc", pos, x['logical_len'], x['hash'], x['tag'])

        primary_map_ptr = f.tell()
        f.write(map_metadata_raw)

        shadow_map_ptr = f.tell()
        f.write(map_metadata_raw)

        f.write(struct.pack(">QQQ32s", primary_map_ptr, shadow_map_ptr, orig_size, master_hasher.digest()))
    else:
        f.write(ram_data)

    f.flush()
    os.fsync(f.fileno())

    os.replace(tmp_path, path)
except Exception as e:
    if os.path.exists(tmp_path): os.remove(tmp_path)
    raise e

end_time = time.perf_counter()
latency = end_time - start_time
throughput = (orig_size / (1024*1024)) / latency if latency > 0 else 0
physical_disk_size = os.path.getsize(path)

print("\n" + "-"*60)
print(f" FILE NAME       : {os.path.basename(path)}")
print(f" ORIGINAL SIZE    : {orig_size:,} bytes")
print(f" AVG ENTROPY      : {avg_ent:.2f}%")
print(f" TIER/MODE        : {tier_name} (≤{tier_limit}%) / {storage_mode}")
print(f" SHIELD PROTOCOL: Achilles Heel (Dual-Map Metadata Mirror)")
print(f" ROUTE TAKEN      : {route}")
print(f" PHYSICAL DISK    : {physical_disk_size:,} bytes (Padded 4KB Sectors)")
if is_efficient:
    gain_pct = (net_saving / orig_size) * 100
    print(f" LOGICAL ECONOMY: {net_saving:,} bytes ({gain_pct:.2f}%)")
print(f" SYSTEM LATENCY   : {latency:.4f} seconds")
print(f" PURE CPU TAX     : {cumulative_cpu_time:.6f} seconds")
print(f" THROUGHPUT       : {throughput:.2f} MB/s")
print("-"*60)

def trigger_open(self, path):
    if not os.path.exists(path): return

```

```
start_time = time.perf_counter()

with open(path, "rb") as f:
    v_data = f.read()

if not v_data.startswith(self.magic_sig): return

try:
    primary_ptr, shadow_ptr, orig_size, m_hash_exp = struct.unpack(">QQQ32s", v_data[-56:])
    num_chunks = (shadow_ptr - primary_ptr) // 45
    master_verify = hashlib.sha256()

    tmp_out = path + ".restored.tmp"
    with open(tmp_out, "wb") as out:
        for i in range(num_chunks):
            idx_primary = primary_ptr + (i * 45)
            idx_shadow = shadow_ptr + (i * 45)
            chunk_valid = False

            # ATTEMPT 1: Primary Map (Wrapped in an absolute blast shield)
            try:
                off, logical_len, hexp, tag = struct.unpack(">QI32sc", v_data[idx_primary:idx_primary+45])
                if tag not in (b'Z', b'R'): raise ValueError()

                raw_payload = v_data[off : off + logical_len]

                if tag == b'Z': chunk = zlib.decompress(raw_payload)
                else: chunk = raw_payload

                if hashlib.sha256(chunk).digest() != hexp: raise ValueError()

                chunk_valid = True
            except Exception:
                pass

            # ATTEMPT 2: The Shadow Map Failover (Taint Protocol)
            if not chunk_valid:
                # Drop the 0-byte Taint Flag instantly
                with open(".vfs_tainted", "w") as tf: pass

                off, logical_len, hexp, tag = struct.unpack(">QI32sc", v_data[idx_shadow:idx_shadow+45])
                raw_payload = v_data[off : off + logical_len]

                if tag == b'Z': chunk = zlib.decompress(raw_payload)
                else: chunk = raw_payload
```

```

        if hashlib.sha256(chunk).digest() != hexp:
            raise ValueError(f"FATAL: Both Primary and Shadow Maps destroyed at chunk {i}")

        master_verify.update(chunk)
        out.write(chunk)

        out.flush()
        os.fsync(out.fileno())
        os.replace(tmp_out, path)

    latency = time.perf_counter() - start_time
    throughput = (orig_size / (1024*1024)) / latency if latency > 0 else 0

    if master_verify.digest() == m_hash_exp:
        print(f"\n[SUCCESS] {path} Restored (Bit-Perfect via Map Shield).")
        print(f" RESTORE LATENCY: {latency:.4f} seconds")
        print(f" RESTORE SPEED : {throughput:.2f} MB/s")
    else:
        print("\n[CRITICAL] Master Integrity Violation!")
except Exception as e:
    if os.path.exists(path + ".restored.tmp"): os.remove(path + ".restored.tmp")
    print(f"\n[CRITICAL FAIL] Restoration Aborted: {e}")

def main():
    vfs = VFSSovereignArchitect()

    while True:
        is_tainted = os.path.exists(".vfs_tainted")

        if is_tainted:
            print("\n" + "="*70)
            print(" [!] RED FLAG STATUS: HARDWARE ROT DETECTED ON PHYSICAL PLATTER")
            print(" [!] SYSTEM HEALING ENGAGED. DATA AT RISK. ")
            print(" [!] PRESS [4] TO CLEAR THIS HARDWARE FLAG AFTER REPLACING DRIVE.")
            print("="*70)
            print(">>> VFS SOVEREIGN SYSTEM ONLINE [STATE: TAINTED] <<<")
            print(" [1] SAVE   [2] OPEN   [3] EXIT   [4] CLEAR FLAG")
        else:
            print("\n>>> VFS SOVEREIGN SYSTEM ONLINE <<<")
            print(" [1] SAVE   [2] OPEN   [3] EXIT")

        cmd = input("Command: ").strip()
        if cmd == "3": break

        if cmd == "4" and is_tainted:
            os.remove(".vfs_tainted")
            print("\n[+] Hardware flag cleared. System returned to pristine state.")

```

```

        continue

    if cmd not in ("1", "2"): continue

    fn = input("Target Filename: ").strip()

    if cmd == "1":
        if not os.path.exists(fn):
            print(f"[!] File '{fn}' not found.")
            continue

        with open(fn, "rb") as f: plasma = f.read()

        print("\n[ DATA ROUTING TIER ]")
        t_choice = input("Select Tier ([1] Enterprise (80% Entropy Threshold) | [2] Executive (70% Entropy Threshold)): ").strip()

        print("\n[ STORAGE MODE ]")
        m_choice = input("Select Mode ([1] HOT (Fast I/O, Level 1 Compress) | [2] COLD (Deep Archival, Level 9 Compress)): ").strip()

        limit = 80.0 if t_choice == "1" else 70.0
        name = "ENTERPRISE" if t_choice == "1" else "EXECUTIVE"
        mode = "HOT" if m_choice == "1" else "COLD"

        vfs.trigger_save(fn, plasma, limit, name, mode)

    elif cmd == "2":
        vfs.trigger_open(fn)

if __name__ == "__main__":
    main()

```

>>> VFS SOVEREIGN SYSTEM ONLINE <<<

[1] SAVE [2] OPEN [3] EXIT

Command: 2

Target Filename: band1.zip

[SUCCESS] band1.zip Restored (Bit-Perfect via Map Shield).

RESTORE LATENCY: 0.0086 seconds

RESTORE SPEED : 10.66 MB/s

```

=====
[!] RED FLAG STATUS: HARDWARE ROT DETECTED ON PHYSICAL PLATTER
[!] SYSTEM HEALING ENGAGED. DATA AT RISK.
[!] PRESS [4] TO CLEAR THIS HARDWARE FLAG AFTER REPLACING DRIVE.
=====

```

>>> VFS SOVEREIGN SYSTEM ONLINE [STATE: TAINTED] <<<

```
[1] SAVE [2] OPEN [3] EXIT [4] CLEAR FLAG
```

```
Command: 4
```

```
[+] Hardware flag cleared. System returned to pristine state.
```

```
>>> VFS SOVEREIGN SYSTEM ONLINE <<<
```

```
[1] SAVE [2] OPEN [3] EXIT
```

```
Command: 1
```

```
Target Filename: band1.zip
```

```
[ DATA ROUTING TIER ]
```

```
Select Tier ([1] Enterprise (80% Entropy Threshold) | [2] Executive (70% Entropy Threshold)): 1
```

```
[ STORAGE MODE ]
```

```
Select Mode ([1] HOT (Fast I/O, Level 1 Compress) | [2] COLD (Deep Archival, Level 9 Compress)): 1
```

---

```
FILE NAME       : band1.zip
ORIGINAL SIZE    : 96,037 bytes
AVG ENTROPY      : 72.42%
TIER/MODE        : ENTERPRISE (≤80.0%) / HOT
SHIELD PROTOCOL : Achilles Heel (Dual-Map Metadata Mirror)
ROUTE TAKEN      : VFS (Vault)
PHYSICAL DISK    : 69,872 bytes (Padded 4KB Sectors)
LOGICAL ECONOMY : 28,210 bytes (29.37%)
SYSTEM LATENCY   : 0.0075 seconds
PURE CPU TAX     : 0.002203 seconds
THROUGHPUT       : 12.23 MB/s
```

---

```
>>> VFS SOVEREIGN SYSTEM ONLINE <<<
```

```
[1] SAVE [2] OPEN [3] EXIT
```

```
Command: 3
```

```
import os
import struct

# The Chaos Monkey: Targets the Primary Map and destroys it.
target_file = "band1.zip" # Change if your file is named differently

with open(target_file, "r+b") as f:
    f.seek(-56, os.SEEK_END) # Go to the dual-pointer footer
    primary_ptr, shadow_ptr, orig_size, m_hash = struct.unpack(">QQQ32s", f.read(56))

    # Seek to the Primary Map and overwrite the first 10 bytes with garbage zeros
    f.seek(primary_ptr)
    f.write(b"\x00" * 10)

print("[SABOTAGE COMPLETE] The Primary Map has been destroyed.")
```

[SABOTAGE COMPLETE] The Primary Map has been destroyed.